Introduction to TensorFlow:-

TensorFlow is an open-source Machine Learning and Deep Learning framework developed by Google. It helps developers build, train, and deploy AI models.

Features

- open-source and scalable
- supports CPU, GPU, and TPU
- Efficient numerical computation
- Easy neural network development through keras

TensorFlow works with tensors (multi-dimensional arrays) and uses data flow graphs to perform computations efficiently on CPUs, GPUs and TPUs.

The framework supports multiple programming languages such as python, javascript, C++ and Java, making it flexible for developers.

It is available on major operating systems including Linux, Windows, macOS, Android and iOS

History of TensorFlow

A few years ago, deep learning algorithms began outperforming traditional machine learning techniques when trained on large amounts of data. Google recognized the potential of deep neural networks and used them to improve services such as:

- Gmail • Google photos • Google search Engine

To support this development, Google created TensorFlow so that researchers and developers could collaborate on AI models. once developed and scaled, these models could be deployed to millions of users.

TensorFlow is released under the Apache open source licence, allowing users to freely use, modify and redistribute the software.

How TensorFlow works:-

TensorFlow works by creating dataflow graphs that describe how data moves through a series of computational operations. The input data is represented as tensors (multidimensional arrays). These tensors flow through different operations in a graph and produces the final output.

The Tensorflow workflow generally consists of three stages:

1. preprocessing the data
2. Building the model
3. Training and evaluating the model

The name TensorFlow comes from the idea that tensors flow through a graph of operations to produce results.

~~Graphs~~ and Nodes in TensorFlow.

TensorFlow represents computations using a computational Graph. A graph is a collection of nodes connected by edges, where the edges represent the flow of data (tensors) between operations. This graph-based approach allows TensorFlow to execute computations efficiently on different hardware platforms.

Graph:-

A graph is a data structure that describes a series of computations. It defines how data moves through different operations and how the final output is produced. In Tensorflow, graphs help organise complex mathematical operations and improves execution performance.

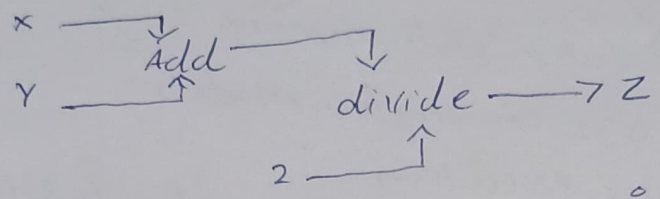
Features of Graphs:-

- Represent the flow of computations
- Consist of nodes and edges.

- Enable efficient execution and optimization
- Allow computations to be distributed across CPUs and GPUs

Eg: $z = (x+y)/2$

This can be represented as,



#In TensorFlow 2.x, eager execution is enabled by default. Graphs can be created using tf.function.

```
import tensorflow as tf
@tf.function
def calculate(x,y):
    return (x+y)/2
print(calculate(2,7))
```

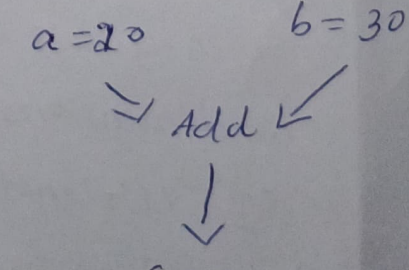
Tensor Flow Nodes:

A node represents an operation or computation in a TensorFlow computational graph. operations represented by nodes include,

- Addition
- Subtraction
- Multiplication
- Activation functions
- Matrix multiplication
- Neural network layers.

Eg:

```
a = tf.constant(20)
b = tf.constant(30)
c = tf.add(a,b)
```



This computation can be viewed as,

#In a neural network, many operations can be represented as nodes.

Types of Nodes:-

1) Constant Nodes - store fixed values
2) Variable Nodes - store values that can change during training.
3) Operation Nodes - perform mathematical operations.

Advantages of Graphs and Nodes

- Efficient computation and memory usage
- Easy visualization of complex models.
- Supports parallel execution.
- Enables deployement on different hardware platforms.

TensorFlow Data Structures.

TensorFlow uses tensors as its primary data structure. A tensor is a multidimensional array that stores data and is characterized by three important properties:,

Rank, shape and Type.

Rank of a Tensor

The Rank of a tensor refers to the no of dimensions (axes) it contains. It indicates the complexity of the tensor structure.

Types of Tensors Based on Rank.

1) Rank 0 Tensor (scalar)

A scalar contains only a single value and has no dimensions.

eg: tf.constant(5)

Rank = 0

Applications
- Age
- Temperature
- single numerical value.

Rank 1 Tensor (vector):-

A vector is a one-dimensional array containing multiple values.

Eg:- tf.constant([10,20,30,40])
Rank=1

Applications
- List of marks
- sensor readings
- Feature values

Rank 2 Tensor (Matrix):-

A matrix is a two-dimensional array arranged in rows and columns.

Eg: tf.constant([[1,2,3],
[4,5,6]])

Rank=2

Applications
- Tables
- Images (grayscale)
- Datasets.

Rank 3 Tensor

A rank-3 tensor contains three dimensions.

Eg: [[[1,2],[3,4]],
[[5,6],[7,8]]]

Rank=3

Applications
- Color images (Height x Width x Channels)
- Video frames
- Deep Learning inputs.

Higher-Rank Tensors

TensorFlow supports tensors with more than three dimensions.

Eg:-

Rank 4 → Batch of images

Rank 5 → Batch of videos.

These are commonly used in deep learning applications.

Checking Rank in TensorFlow

```
import tensorflow as tf
x = tf.constant([[1,2],[3,4]])
print(tf.rank(x))
```

Output : 2.

Shape

Shape specifies the size of each dimension of a tensor. It describes how the data is arranged.

Tensor	shape
5	()
[1,2,3]	(3,)
[[1,2],[3,4]]	(2,2)
[[[1],[2]],[[3]][4]]]	(2,2,1)

shape shows the number of elements in each dimension.
A matrix with 2 rows and 3 columns has shape (2,3)
shape helps TensorFlow understand the structure of data.

Type (dtype)

Type (dtype) specifies the kind of values stored in a tensor. TensorFlow supports various data types.

Data Type	Description
Int32	32-bit integer
Int64	64-bit integer
Float32	32-bit floating-point number
float64	64-bit floating-point number
String	Text data
bool	True or False values

Type determines the nature of stored data.
Choosing the correct dtype improves memory efficiency and performance

• Metrics - Eval...

Building Neural Networks with TensorFlow:

TensorFlow provides the Keras API, a high-level interface that makes it easy to build, train, and evaluate neural networks. A neural network consists of layers of interconnected neurons that learn patterns from data and make predictions.

Steps to Build a Neural Network using TensorFlow

1) Import TensorFlow Library

```
import tensorflow as tf
```

2) Create the Neural Network Model

TensorFlow uses the sequential model to stack layers one after another.

```
model = tf.keras.Sequential()
```

3) Add layers.

A neural network typically consists of

- Input layer - Receives input data.
- Hidden Layer - Learns patterns and features.
- Output Layer - produces the final result.

```
model = tf.keras.Sequential([tf.keras.layers.Dense(16, activation='relu', input_shape=(4,)), tf.keras.layers.Dense(8, activation='relu'), tf.keras.layers.Dense(3, activation='softmax')])
```

4) Compile the Model

Compilation specifies

- optimizer - updates weights during training
- Loss Function - Measures prediction error
- Metrics - Evaluates model performance.

```
model.compile ( optimizer = 'adam',
                loss = 'categorical_crossentropy', metrics = ['accuracy']
              )
```

5) Train the model

The model learns from the training data.

```
model.fit (x_train, y_train, epochs = 10)
```

x_train → Input data

y_train → Target Output

epochs → Number of training cycles.

6) Evaluate the Model

After training, evaluate the model's performance

```
loss, accuracy = model.evaluate (x_test, y_test)
print ("Accuracy:", accuracy)
```

Advantages of TensorFlow For Neural Networks:

- Easy model creation using keras API
- support deep learning applications
- Efficient execution on CPUs and GPUs
- Automatic differentiation and optimization
- suitable for image processing, NLP, speech recognition and prediction tasks.